



VIT[®]
BHOPAL
www.vitbhopal.ac.in

***Programming in Java - Evaluated
Project***

Medicare

**PROGRAMMING IN JAVA
CSE2006 (B21+E14)**

**BY Anushka Sahu
25BAI11314**

**Dr. Praveen Kumar Tyagi
(Faculty)**

Introduction

Medicare – Hospital/Clinic Appointment, Prescription & Billing Management System is a Java-based application developed as part of the Vidyarthi *Build Your Own Project* for the Programming in Java course. The project aims to apply Java programming concepts to a practical healthcare-management scenario. The Vidyarthi guidelines emphasize identifying a meaningful problem, designing a technical solution, implementing it using course concepts, and demonstrating the solution through documentation and evaluation.

Medicare provides a centralized system for managing essential clinic operations such as patient management, doctor management, appointment scheduling, prescription management, billing, and appointment reminders. It is designed as a terminal-based application, allowing users to interact with the system through a simple menu-driven interface.

The project demonstrates important Java concepts including Object-Oriented Programming, classes and objects, encapsulation, inheritance, abstraction, collections, exception handling, input validation, regular expressions, synchronization, and multithreading. The system also incorporates business rules such as preventing doctor double-booking and validating user inputs.

Through this project, the aim is to demonstrate how fundamental Java concepts can be combined to develop a structured, modular, and practical application for solving a real-world management problem.

Problem Statement

In hospitals and clinics, managing patient records, doctor information, appointments, prescriptions, and billing manually can be time-consuming and difficult to organize. Manual management may result in duplicate records, appointment scheduling conflicts, difficulty in retrieving information, and errors while handling important details.

To address these issues, MediCare is developed as a Java-based hospital/clinic management system that provides a centralized way to manage these essential operations. The system allows users to add and manage patient and doctor records, book and manage appointments, create prescriptions, generate bills, and provide appointment reminders.

A major objective of the system is to provide a structured workflow in which user input is validated before processing and appropriate business rules are applied. For example, the system checks doctor availability before confirming an appointment, helping prevent double-booking.

The project also provides an opportunity to apply important Programming in Java concepts, including Object-Oriented Programming, collections, exception handling, validation, regular expressions, synchronization, and multithreading. This aligns with the VITyarthi requirement for a meaningful problem, a technical solution, and implementation using concepts learned in the course.

Functional Required

The MediCare system shall provide essential clinic-management functions including patient management, doctor management, appointment management, prescription management, and billing. Users can add, view, search, and manage patient and doctor records; book, view, cancel, and reschedule appointments; create prescriptions with multiple medicines; and generate bills with payment status. The system shall also check doctor availability before confirming an appointment to prevent double-booking.

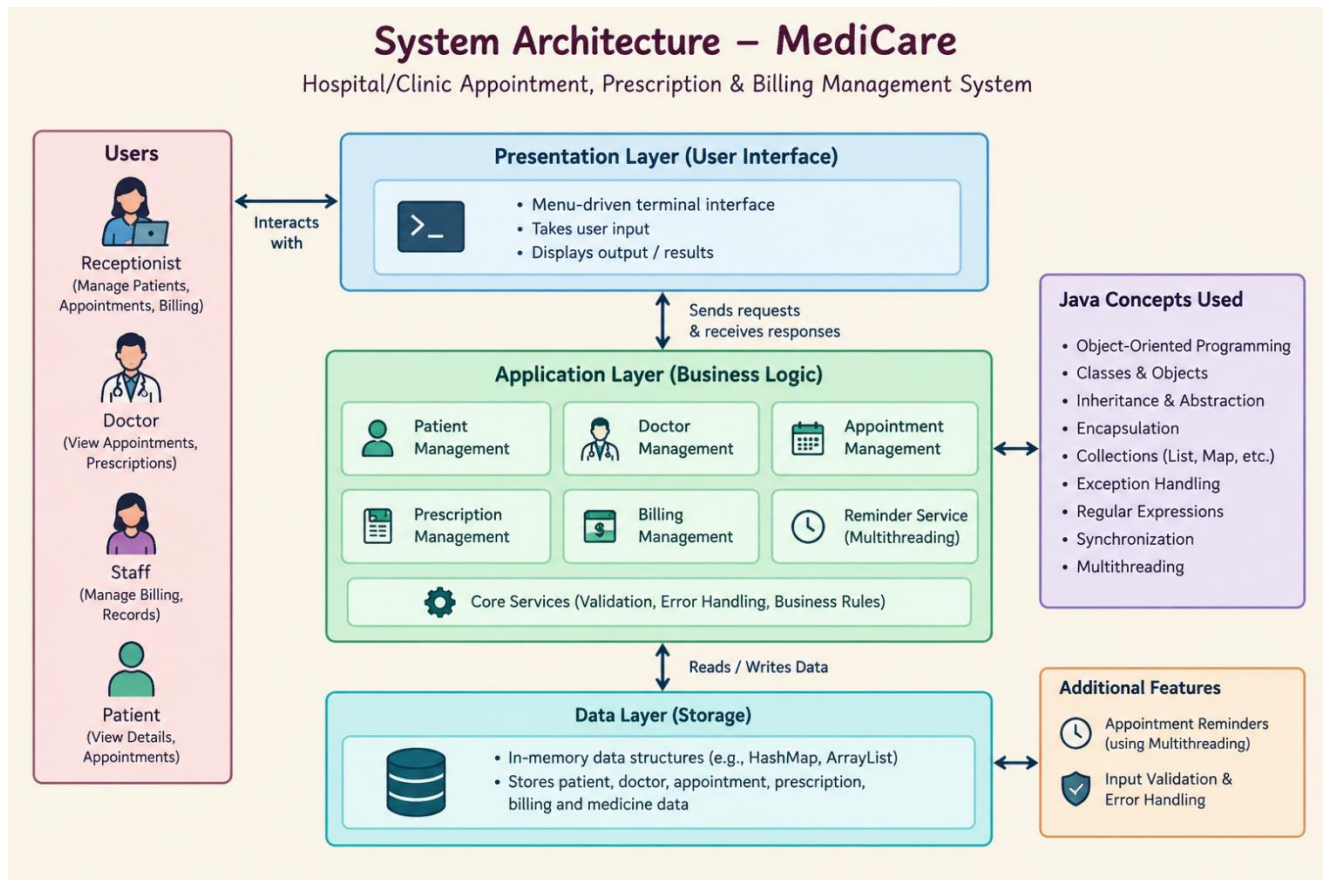
The system shall accept user input through a menu-driven terminal interface and produce appropriate outputs such as confirmations, records, appointment status, prescription details, and bills. It shall validate user inputs, handle invalid operations and exceptions, and provide meaningful error messages. Appointment reminders shall also be processed using Java multithreading concepts. These functions satisfy the requirement for multiple major functional modules with clear input/output and a logical user workflow.

Non – Functional Requirement

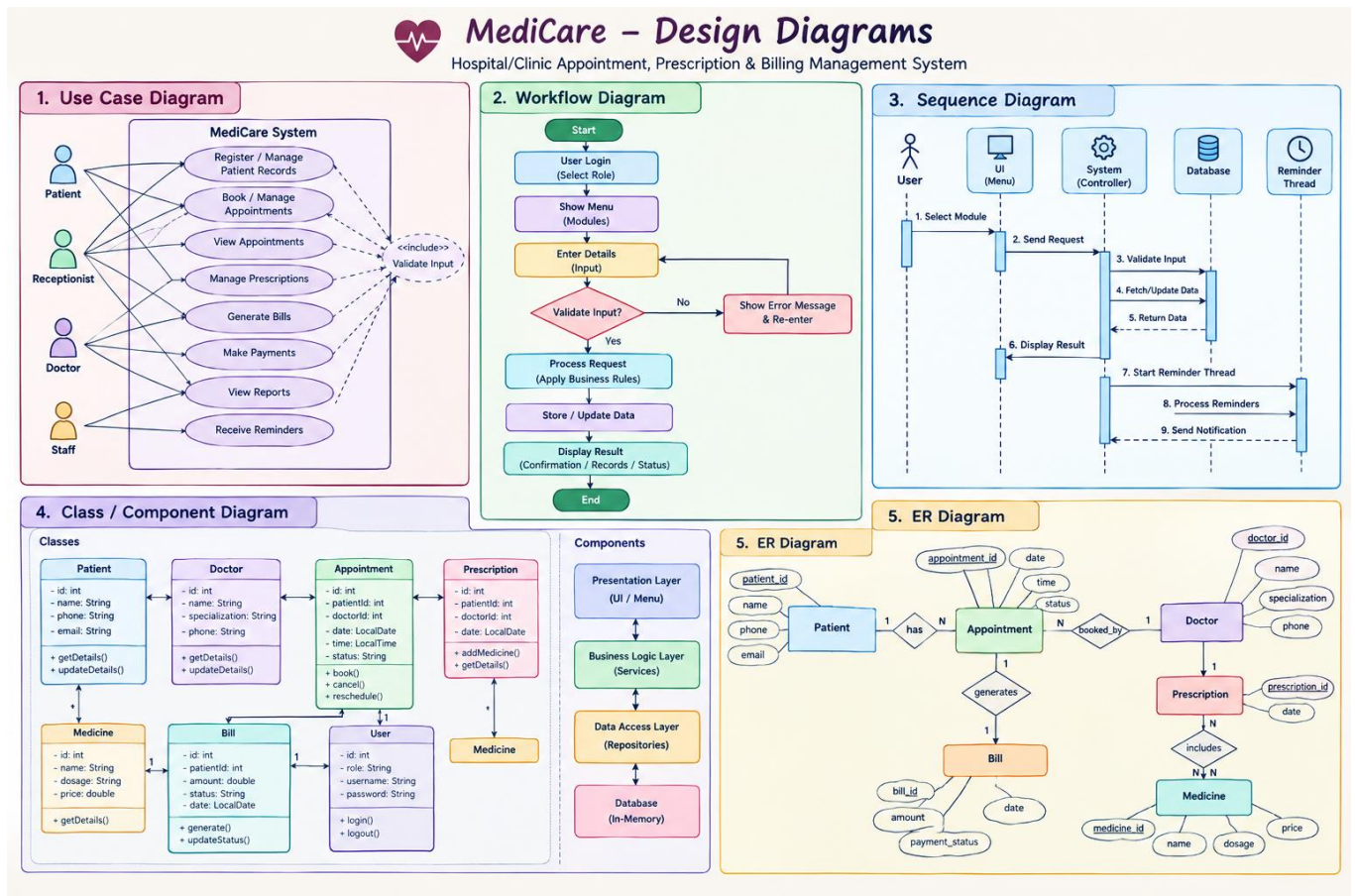
The MediCare system shall provide usability, performance, reliability, security, and maintainability to ensure that the application works efficiently and consistently. The system should have a simple menu-driven interface, respond quickly to user operations, validate inputs, and handle errors without unexpected termination. It should also maintain data consistency while managing records and appointments.

The system shall be designed using modular and maintainable Java code, with proper exception handling and synchronization where required. It should efficiently manage collections and system resources, while restricting operations to valid inputs and maintaining reliable application behavior. These requirements focus on the quality, efficiency, security, reliability, and maintainability of the system rather than specific features.

System Architecture



Design Diagrams



Design Decisions & Rationale

The MediCare system was designed as a menu-driven, terminal-based Java application to keep the interface simple and easy to use. The system follows a modular structure in which patient management, doctor management, appointments, prescriptions, billing, and reminders are handled separately. Object-Oriented Programming, encapsulation, inheritance, abstraction, collections, exception handling, regular expressions, synchronization, and multithreading were selected to make the implementation structured and demonstrate the concepts covered in Programming in Java.

The project uses in-memory data structures such as ArrayList and HashMap instead of an external database, keeping the application lightweight and focused on Java concepts. Input validation and exception handling were included to prevent invalid data and unexpected program termination. Appointment availability checking and synchronization help maintain consistent appointment records, while Thread/Runnable is used for appointment reminders. These design choices provide a clear workflow, maintainable code, and practical implementation of the project's functional requirements.

Implementation Details

MediCare is implemented as a Java-based, terminal-driven application using standard Java features. The system contains modules for patient management, doctor management, appointment management, prescription management, billing, and appointment reminders. Collections such as ArrayList and HashMap are used to manage records in memory, while classes and objects are used to represent entities such as patients, doctors, appointments, prescriptions, medicines, and bills.

The implementation applies important Java concepts including encapsulation, inheritance, abstraction, exception handling, input validation, regular expressions, synchronization, and multithreading. User inputs are validated before processing, and business rules such as checking doctor availability are applied during appointment operations. The reminder functionality uses Thread/Runnable, while synchronization helps maintain consistency when shared data is accessed. The application can be compiled using `javac MediCare.java` and executed using `java MediCare`.

OUTPUT

```
*****
*      Welcome to MediCare Hospital System      *
*****

MAIN MENU:
[System] Background Reminder Thread Started...
1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing & Payments
6. Exit
Choose option: 1

[PATIENT MANAGEMENT]
1. Add Patient  2. View All  3. Search  4. Delete
1
Name: Rahul Sharma
Age: 25
Phone: 9876543210
Email: rahul@gmail.com
Address: Sector 62, Noida
Patient added successfully.
```

MAIN MENU:

1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing & Payments
6. Exit

Choose option: 2

[DOCTOR MANAGEMENT]

1. Add Doctor
2. View All

1

Name: Dr. Amit Verma

Specialization: Cardiology

Phone: 8877665544

Doctor added.

MAIN MENU:

1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing & Payments
6. Exit

Choose option: 1

[PATIENT MANAGEMENT]

1. Add Patient
2. View All
3. Search
4. Delete

2

[ID: 1] Name: Rahul Sharma | Age: 25 | Phone: 9876543210

MAIN MENU:

1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing & Payments
6. Exit

Choose option: 2

[DOCTOR MANAGEMENT]

1. Add Doctor
 2. View All
- 2

[ID: 1] Dr. Dr. Amit Verma | Specialization: Cardiology

MAIN MENU:

1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing & Payments
6. Exit

Choose option: 3

[APPOINTMENT MANAGEMENT]

1. Book
 2. View All
 3. Cancel
- 1

Patient ID: 1

Doctor ID: 1

Date (YYYY-MM-DD): 2026-09-17

Time (HH:MM): 10:30

Appointment Booked.

```
MAIN MENU:
1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing & Payments
6. Exit
Choose option: 4

[PRESCRIPTION MANAGEMENT]
1. Create Prescription 2. View All

>>> REMINDER: Appointment #1 for Rahul Sharma is today at 10:30!

Error: For input string: ""
```

```
MAIN MENU:
1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing & Payments
6. Exit
Choose option:
>>> REMINDER: Appointment #1 for Rahul Sharma is today at 10:30!
5

[BILLING MANAGEMENT]
1. Generate Bill 2. View Bills
1
Appointment ID: 1
>>> REMINDER: Appointment #1 for Rahul Sharma is today at 10:30!
```

```
MAIN MENU:
1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing & Payments
6. Exit
Choose option:
>>> REMINDER: Appointment #1 for Rahul Sharma is today at 10:30!
5

[BILLING MANAGEMENT]
1. Generate Bill 2. View Bills
1
Appointment ID: 1
>>> REMINDER: Appointment #1 for Rahul Sharma is today at 10:30!

Consultation Fee: 500
Medicine Charges: 600
Mark as Paid? (y/n): y
Bill Generated: [Bill ID: 1] Total: ₹1100.00 | Status: Paid
```

TESTING APPROACH

The MediCare system is tested by performing **functional and input-validation testing** on each major module. Patient and doctor records are tested by adding, viewing, searching, and updating data. Appointment management is tested for booking, cancellation, rescheduling, and prevention of **doctor double-booking**. Prescription and billing functions are tested by creating records and checking whether the correct information is displayed. Appointment reminders are also tested to verify the multithreading functionality.

Invalid inputs and incorrect operations are deliberately provided to check the system's **validation and exception handling**. Test cases include empty or incorrectly formatted details, unavailable appointment slots, invalid record IDs, and incorrect menu choices. The expected output is compared with the actual output to identify errors and verify that the system performs its required operations correctly.

Challenges Faced

During the development of MediCare, one of the main challenges was organizing multiple healthcare-related operations within a single Java application. Managing patient, doctor, appointment, prescription, and billing data required careful design of classes, collections, and relationships between records. Implementing appointment scheduling was also challenging because the system needed to validate inputs and prevent doctor double-booking while maintaining correct appointment information.

Another challenge was implementing exception handling, input validation, synchronization, and multithreading correctly. The reminder feature required understanding Java's Thread/Runnable concepts, while synchronization was needed when working with shared data. Handling incorrect user input without terminating the program also required additional validation and error-handling logic. These challenges helped improve understanding of Java programming concepts and their practical application.

Learnings & Key Takeaways

Developing the MediCare project provided practical experience in applying Programming in Java concepts to a real-world problem. The project helped in understanding how Object-Oriented Programming, classes and objects, encapsulation, inheritance, abstraction, collections, exception handling, regular expressions, and input validation can be combined to build a structured application.

The project also provided hands-on learning in multithreading and synchronization through the appointment reminder and shared-data handling. Designing different modules and implementing business rules such as preventing appointment conflicts improved understanding of logical problem-solving and software design. Overall, the project strengthened Java programming skills and demonstrated how theoretical concepts can be applied to develop a meaningful technical solution.

Future Enhancements

In the future, MediCare can be enhanced by adding **persistent database storage** so that patient, doctor, appointment, prescription, and billing records are retained even after the application is closed. The system could also be extended with a **graphical or web-based interface**, user authentication, role-based access, and improved security for protecting sensitive records.

Additional features such as **online appointment booking, automated SMS/email reminders, digital prescriptions, payment integration, report generation, and advanced search and analytics** can also be introduced. The application could further be made scalable by separating its components into independent modules and connecting it to a centralized database. These enhancements would extend the current academic implementation into a more comprehensive healthcare-management system.

Conclusion

The MediCare – Hospital/Clinic Appointment, Prescription & Billing Management System demonstrates how Java programming concepts can be applied to develop a practical solution for managing essential clinic operations. The project provides functionality for patient and doctor management, appointment scheduling, prescriptions, billing, and appointment reminders, while also incorporating input validation and error handling.

Through this project, important Programming in Java concepts such as Object-Oriented Programming, collections, exception handling, regular expressions, synchronization, and multithreading were applied in a practical context. The development process also improved understanding of software design, logical problem-solving, and modular implementation. Overall, MediCare fulfills the objective of creating a meaningful technical solution while demonstrating the practical application of concepts learned in the course.